PGBD - PROJET DE DÉVELOPPEMENT SGBD

MAUI – Data Binding

Frédéric Parion
f.parion@esa-namur.be

1

Plan

- Concepts de base
  - Définition
  - Classe Binding
  - Propriétés de la classe Binding
  - Notion de BindingContext
  - Binding sur une collection
- Notion de ViewModel
- DataBinding en lien avec une base de données
- Notifications de changement

2

2

DATA BINDING MAUI

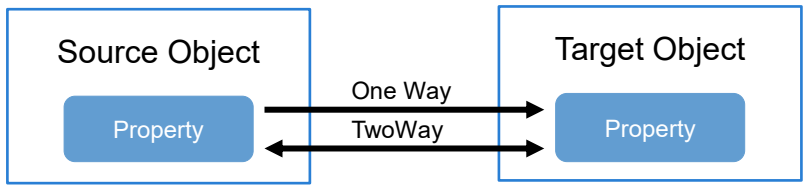
Concepts de base

3

Data Binding

Définitions

Le **Binding** (lien ou liaison) est l'acte de lier/connecter les propriétés de deux objets de façon à ce qu'une modification de la propriété sur l'objet source entraine automatiquement la modification de la propriété sur l'objet destination (voire dans les deux sens).



Dans les interfaces utilisateurs, le **Data Binding** (liaison de données) est l'acte de lier/connecter les propriétés d'un ou de plusieurs contrôles d'interface utilisateur (Windows ou Web) à une source de données.

4

4

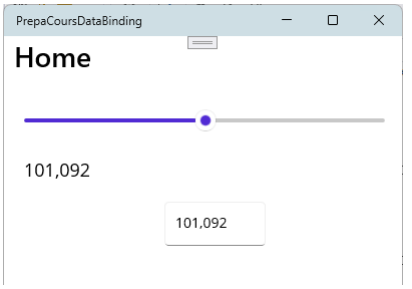
Data Binding

Exemple

```
<Slider x:Name="TestSlider" Minimum="1"
Maximum="200" Value="1" Margin="20,20,20,0"/>

<Label Text="{Binding Source={x:Reference
TestSlider}, Path=Value}" Margin="20,20,0,0"
FontSize="18"/>

<Entry Text="{Binding Source={x:Reference
TestSlider}, Path=Value}"
WidthRequest="{Binding Source={x:Reference
TestSlider}, Path=Value}"
Margin="20,20,0,0"/>
```



5

5

Data Binding

Classe Binding

- Le data binding est mis en œuvre grâce à la classe **Binding**
- Une instance de Binding permet d'établir et de définir une relation entre une propriété destination et une propriété source.
- Deux façon de définir un binding:
  - De **façon déclarative** dans le XAML:
    ```
    <Label Text="{Binding Source={x:Reference TestSlider}, Path=Value}" />
    ```
  - Dans le **code**:
    ```
    Binding sliderBinding = new Binding();
    sliderBinding.Source = TestSlider;
    sliderBinding.Path = "Value";
    MyLabel.SetBinding(Label.TextProperty, sliderBinding);
    ```

6

6

Data Binding

Propriétés de la classe Binding

Propriété	Description
Source	Objet qui constitue la source de la liaison
Path	Propriété qui constitue la source (dans l'objet source)
Mode	Direction de la liaison: OneWay, TwoWay, OneTime
StringFormat	Définition d'un chaine de format (alternative au convertisseur)
Converter	Définition d'un convertisseur à utiliser quand le type de la propriété source et de la propriété destination ne sont pas compatibles
ConverterParameter	Paramètre optionnel pour le convertisseur

```
Binding sliderBinding = new Binding();
sliderBinding.Source = TestSlider;
sliderBinding.Path = "Value";
sliderBinding.Mode = BindingMode.TwoWay;
sliderBinding.StringFormat = "{0:0}";
MyLabel.SetBinding(Label.TextProperty, sliderBinding);
```

7

7

Data Binding

Exemple de convertisseur: ScoreToColorConverter (1/2)

```
public class ScoreToColorConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
    {
        int score;
        if (int.TryParse((string)value, out score))
        {
            if (score < 50) return Colors.Red;
            else if (score < 80) return Colors.Orange;
            else return Colors.Green;
        }
        return Colors.Transparent; // Valeur par défaut si le binding échoue
    }

    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
    {
        throw new NotImplementedException(); // Pas de conversion inverse nécessaire
    }
}
```

8

8

Data Binding

Exemple de convertisseur : ScoreToColorConverter (2/2)

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:converters="clr-namespace:PrepaCoursDataBinding.Converters"
             x:Class="PrepaCoursDataBinding.MainPage" WidthRequest="400" >

    <ContentPage.Resources>
        <ResourceDictionary>
            <converters:ScoreToColorConverter x:Key="ScoreToColorConverter"/>
        </ResourceDictionary>
    </ContentPage.Resources>
 <VerticalStackLayout >

 <Entry x:Name="ScoreEntry" Keyboard="Numeric" Placeholder="Entrez un score" FontSize="18"/>

    <BoxView HeightRequest="50" WidthRequest="50" CornerRadius="10"
           BackgroundColor="{Binding Source={x:Reference ScoreEntry}, Path=Text, Converter={StaticResource
ScoreToColorConverter}}"/>

    </VerticalStackLayout>
</ContentPage>
```

9

9

Data Binding

Notion de BindingContext (1/4)

- Le **BindingContext** est le concept clé du data binding en MAUI car il permet de définir la source des données pour les bindings.
- C'est une propriété qui peut être positionnée au niveau :
  - d'une page (`ContentPage`)
  - d'un contrôle (`Entry`, `Label`, ...)
  - ou d'un layout (`VerticalStackLayout`, ...).

=> toutes les classes qui dérivent de la classe de base **BindableObject**.

10

10

Data Binding

Notion de BindingContext (2/4)

Exemple 1 : BindingContext positionné sur une propriété simple

```
public partial class MainPage : ContentPage
{
    public string WelcomeMessage { get; set; }

    public MainPage()
    {
        InitializeComponent();

        WelcomeMessage = "Bienvenue dans la page test du databinding MAUI";
        WelcomeLabel.BindingContext = WelcomeMessage;
    }
}

<Label x:Name="WelcomeLabel" Text="{Binding}"></Label>
```

11

11

Data Binding

Notion de BindingContext (3/4)

Le BindingContext est hérité de façon descendante dans la hiérarchie des contrôles. On peut donc le positionner au niveau que l'on souhaite, même au niveau de la page.

Exemple 2 : BindingContext positionné sur un objet personne

```
public class Personne
{
    public string LastName { get; set; }
    public string FirstName { get; set; }
}
```

12

12

Data Binding

Notion de BindingContext (4/4)

XAML:

```
<VerticalStackLayout x:Name="MainLayout" >
    <Label Text="{Binding FirstName}"></Label>
    <Label Text="{Binding LastName}"></Label>
</VerticalStackLayout>
```

BindingContext positionné sur le contrôle de layout

```
Personne unePersonne = new Personne() { FirstName = "Thierry", LastName = "Libert" };
MainLayout.BindingContext = unePersonne;
```

BindingContext positionné sur la page:

```
Personne unePersonne = new Personne() { FirstName = "Thierry", LastName = "Libert" };
this.BindingContext = unePersonne;
```

13

13

Data Binding

Binding sur une collection (1/2)

Les contrôles de type « collection » comme ListView ou CollectionView disposent d'une propriété **ItemsSource** qui permet le binding à une collection d'objets.

<u>Exemple:</u>

```
public List<Personne> Personnes { get; set; }

Personnes = new List<Personne>();
Personnes.Add(unePersonne);
Personnes.Add(new Personne() { FirstName = "Jeanne", LastName = "Dupuis" });

MyCollectionView.ItemsSource = Personnes;
```

14

14

Data Binding

Binding sur une collection (2/2)

Exemple (suite):

```
<CollectionView x:Name="MyCollectionView">
    <CollectionView.ItemTemplate>
        <DataTemplate>
            <StackLayout Padding="10" Orientation="Horizontal">
                <Label Text="{Binding FirstName}" FontSize="18" />
                <Label Text="{Binding LastName}" FontSize="18"
Margin="10,0,0,0" TextColor="Gray"/>
            </StackLayout>
        </DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>
```

15

15

DATA BINDING MAUI

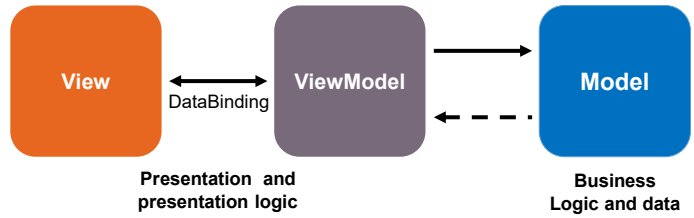
Notion de ViewModel

16

Notion de ViewModel

Concept de ViewModel issu de MVVM (1/2)

- Dans la couche d'affichage MAUI classique, l'apparence est décrite dans le fichier XAML et le fichier code-behind mélange la gestion des interactions (évènements), les calculs et l'accès aux données.
- L'architecture **MVVM**: Model-View-ViewModel a été conçue pour améliorer l'organisation du code dans la couche de présentation en découplant mieux le code UI. Elle a introduit le concept de **ViewModel**.

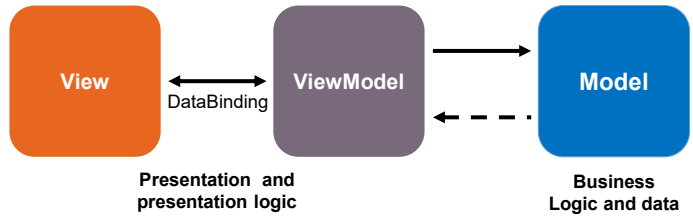


17

17

Notion de ViewModel

Concept de ViewModel issu de MVVM (2/2)



- **View**: Ne s'occupe que de présenter les données et gérer les interactions. XAML + code-behind réduit (limité à gestion de l'affichage)
- **Model**: Logique business et accès aux données
- **ViewModel** : constitue le modèle adapté à la vue
  - Centralise toutes les sources de **données** nécessaires à la vue (Data Binding)
  - Gère les **commandes** en provenance de la vue (suite aux interactions des utilisateur) et décides des actions à réaliser: modifications dans l'interface, chargement ou enregistrement de données, appel de services, ...

18

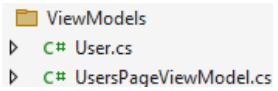
18

Notion de ViewModel

Premier niveau d'utilisation du concept de ViewModel

- Architecture MVVM présente un certain niveau de complexité
- Projet: premier pas vers l'architecture MVVM avec ViewModel simplifié
- Au lieu de gérer différentes sources de données dans le code-behind et de positionner le BindingContext sur plusieurs contrôles de la vue, on va utiliser une classe **ViewModel** pour centraliser toutes les sources de données nécessaires au rendu de la vue.
- Le BindingContext de chaque page sera associée à un ViewModel
- Exemple:

```
public class UsersPageViewModel
{
    public string PageTitle { get; set; }
    public List<User> Users { get; set; }
    public int UserCount => Users.Count;
}
```



19

19

Notion de ViewModel

Exemple: code xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:viewmodels="clr-namespace:PrepaCoursDataBinding.ViewModels"
             x:Class="PrepaCoursDataBinding.UsersPage"
             x:DataType="viewmodels:UsersPageViewModel"
             Title="{Binding PageTitle}">
    <VerticalStackLayout>
        <Label Text="{Binding PageTitle}" FontSize="Header" />
        <HorizontalStackLayout>
            <Label Text="Nombre: "></Label> <Label Text="{Binding UserCount}"></Label>
        </HorizontalStackLayout>
        <CollectionView x:Name="UserCollectionView" ItemsSource="{Binding Users}">
            <CollectionView.ItemTemplate>
                <DataTemplate x:DataType="viewmodels:User">
                    <StackLayout Padding="10" Orientation="Horizontal">
                        <Label Text="{Binding FirstName}"  />
                        <Label Text="{Binding LastName}" />
                    </StackLayout>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </VerticalStackLayout>
</ContentPage>
```

<= déclaration namespace pour accéder au VM

<= définition type du BindingContext de la page (permet l'intellisense)

<= définition type du DataTemplate (type des items de la source)

20

20

Notion de ViewModel

Exemple: code-behind

```
public partial class UsersPage : ContentPage
{
    ViewModels.UsersPageViewModel vm { get; set; }    <= déclaration du viewmodel de la page

    public UsersPage()
    {
        InitializeComponent();
        PopulateAndBind();
    }

    private void PopulateAndBind()    <= méthode de chargement  des données et du binding des contrôles
    {
        vm = new ViewModels.UsersPageViewModel();
        vm.PageTitle = "Liste des utilisateurs";
        vm.Users = new List<User>();
        vm.Users.Add(new User() { FirstName ="Thierry", LastName = "Libert"});
        vm.Users.Add(new User() { FirstName = "Arnaud", LastName = "Guillaume" });

        this.BindingContext = vm;    <= on positionne le BindingContext avec le viewmodel
    }
}
```

21

21

DATA BINDING MAUI

DataBinding en lien avec la base de données

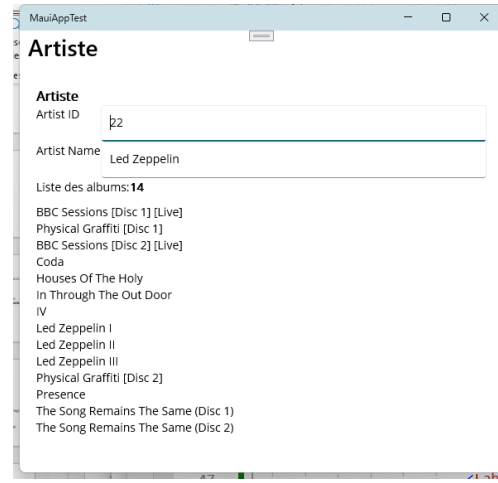
22

Liens avec la base de données

DataBinding en lien avec la base de données

Exemple WorkshopArchi5

(détails voir code)

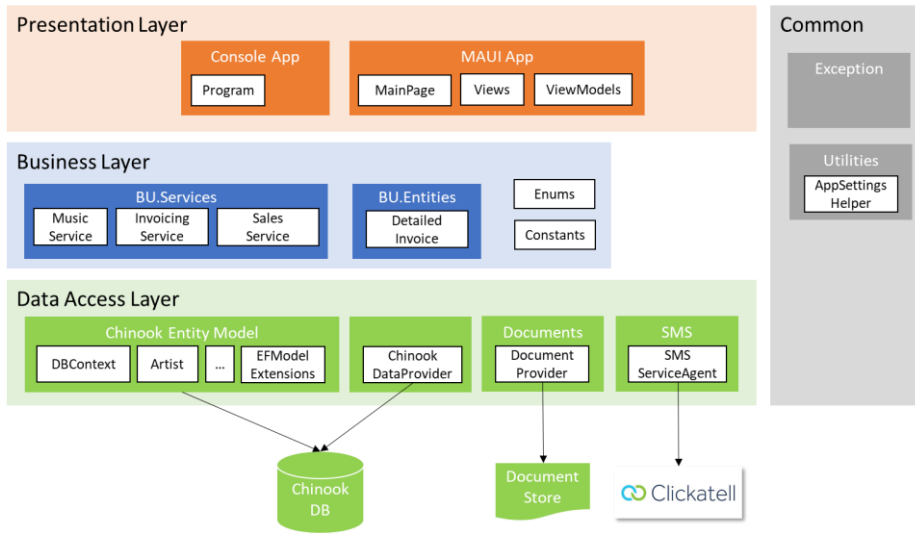


23

23

Liens avec la base de données

Evolution de l'architecture du Workshop



24

24

DATA BINDING MAUI

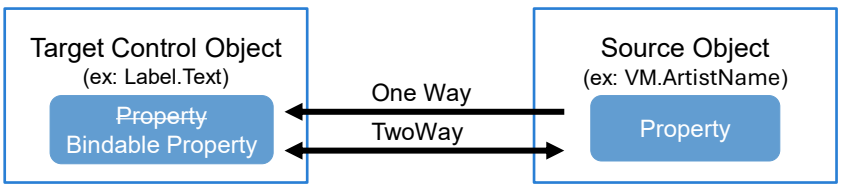
Notifications de changement

25

Notifications de changement

Binding d'une propriété simple : constat

- Les propriétés des contrôles MAUI (ex: Label.Text) ne sont pas des propriétés classiques. Ce sont des BindableProperty (= concept de Dependency Property de WPF).



- Pour qu'il y ait mise à jour de l'UI il faut implémenter une **notification**.
- Possible via l'implémentation de l'interface **INotifyPropertyChanged**

26

26

Notifications de changement

Interface INotifyPropertyChanged (1/3)

- Interface qui a un seul membre: l'évènement **PropertyChanged**

```
namespace System.ComponentModel
{
    // Notifies clients that a property value has changed.
    public interface INotifyPropertyChanged
    {
        // Occurs when a property value changes.
        event PropertyChangedEventHandler PropertyChanged;
    }
}
```

27

27

Notifications de changement

Interface INotifyPropertyChanged (2/3)

- La classe utilisée comme binding context dans le data binding va implémenter l'interface INotifyPropertyChanged.

```
internal class AlbumDetailsPageViewModel : INotifyPropertyChanged
{
    // Declare the event
    public event PropertyChangedEventHandler PropertyChanged;

    // Create the NotifyPropertyChanged method to raise the event
    // The calling member's name will be used as the parameter.
    protected void NotifyPropertyChanged([CallerMemberName] string name = null)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
    }
}
```

- En notifiant que la valeur d'une propriété d'un certain nom à changé, le contrôle d'interface (ex: textbox) « bindé » à la propriété sera informé et pourra rafraîchir son affichage.

28

28

Notifications de changement

Interface INotifyPropertyChanged (3/3)

```
internal class AlbumDetailsPageViewModel : INotifyPropertyChanged
{
    private string artistName;
    public string ArtistName
    {
        get => artistName;
        set
        {
            artistName = value;
            NotifyPropertyChanged();
        }
    }
    public event PropertyChangedEventHandler PropertyChanged;

    protected void NotifyPropertyChanged([CallerMemberName] string name = null)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
    }
}
```

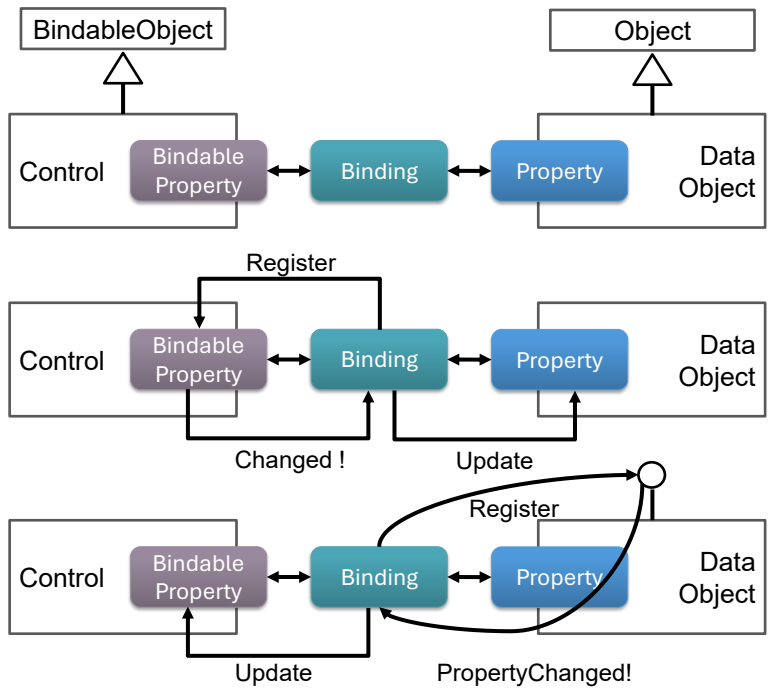
29

29

Résumé

Inspiration:
Peter-Himschoot

https://blogs.u2u.be/peter/post/Using-Model-View-ViewModel-with-WPF



30

30

Notifications de changement

Binding d'une collection d'objets : constat

- Si on lie la propriété ItemsSource d'une CollectionView à une collection d'objets, l'ajout d'un objet dans la collection après le binding n'entraine pas la mise à jour de la CollectionView.
- Pour qu'il y ait mise à jour de l'UI il faut implémenter une **notification**.
- Possible en utilisant une ObservableCollection

```
public ObservableCollection<Track> Tracks { get; set; }
```

```
vm = new AlbumDetailsPageViewModel
{
    AlbumId = albumId,
    AlbumTitle = album.Title,
    ArtistName = album?.Artist.Name,
    Tracks = new ObservableCollection<Track>(album.Tracks)
};
```

31

31

Notifications de changement

BaseViewModel

```
internal class BaseViewModel : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler PropertyChanged;

    protected void NotifyPropertyChanged([CallerMemberName] string name = null)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
    }
}
internal class AlbumDetailsPageViewModel : BaseViewModel
{
    private string artistName;
   public string ArtistName
   {
       get => artistName;
       set
       {
           artistName = value;
           NotifyPropertyChanged();
       }
   }
}
```

<= On peut éviter la répétition de l'implémentation de l'interface en utilisant une classe de base qui q'en charge

<= les classes « ViewModel » héritent de BaseViewModel

32

32

Notifications de changement

MVVM Toolkit

Une alternative à l'implémentation manuelle de l'interface INotifyPropertyChanged est d'utiliser le MVVM Toolkit.

https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm-community-toolkit-features

Avertissement:
Le MVVM Toolkit est prévu pour l'implémentation complète d'une architecture MVVM qui dépasse le cadre du cours de PGBD. Sa prise en main requiert un investissement personnel et une compréhension suffisante pour pouvoir être utilisé dans le cadre du projet.

33

33

Implémentation d'une page du projet PGBD

Objectif du cours du 09/03 en distanciel

Implémenter la page **PropertyPage** du projet pour expérimenter le data binding et la notion de ViewModel tout en exploitant l'architecture en couches de l'application.

=> Détails voir devoir Teams

Je serai disponible en ligne le 09/03 de 18 à 19h00 via Teams pour répondre aux questions.

34

34

Questions ?

35